

Name:T Nithish Kumar (192225094):

Snowflake Healthcare Architecture & Implementation Guide

****Project****: Enterprise Healthcare Analytics & Patient Insights

This document serves as the comprehensive explanation for the advanced Enterprise Data Warehouse architecture implemented in `healthcare\_advanced\_pipeline.sql`.

> [!IMPORTANT]

> You can easily save this explanation as a PDF! Simply right-click on this page in your browser or Markdown viewer, and select ****"Print" -> "Save as PDF"****. This is perfect to hand out during an interview or presentation.

1. Architectural Paradigm

The project utilizes a highly automated, production-grade ****Medallion Architecture****, adapted with advanced Data Governance and Data Mart layers. The pipeline operates entirely within Snowflake using native capabilities.

```
```mermaid
flowchart LR
 CSV[CSV Files] --> |Internal Stage| RAW
 RAW --> |Streams + Task 1| VALIDATED
 VALIDATED --> |Task 2| CURATED
 CURATED --> |Task 3| GOVERNANCE
 GOVERNANCE --> DATAMART
 DATAMART --> ANALYTICAL
```
```

Layer Breakdown

1. ****`RAW` (Bronze)****: Untouched data ingested directly from source CSVs via `COPY INTO`.
2. ****`VALIDATED` (Silver)****: Clean, typed data. Bad records are quarantined.
3. ****`CURATED` (Gold)****: Dimensional models (SCD-2) and Surrogate Keys.
4. ****`GOVERNANCE`****: A secondary auditing layer tracking business anomalies.
5. ****`DATAMART`****: Business-facing views solving specific reporting requirements (Patient 360, Revenue).
6. ****`ANALYTICAL`****: C-Suite level KPI views.

2. Ingestion & Automation (RAW)

****The "How"**: Data is ingested via an internal `@external\_stages.healthcare\_stage`.**

****The "Engine"**: We use Snowflake ****Streams**** (`patients\_stream`, `appointments\_stream`) placed directly on the RAW tables.**

- ***Why Streams?*** Streams enable Change Data Capture (CDC). Instead of scanning the whole table every hour, the pipeline only processes the **delta** (newly inserted rows). This drastically reduces compute costs and enables real-time micro-batching.

3. Data Quality & Cleaning (VALIDATED)

Before data ever touches a business report, it passes through `validated.process\_data\_quality()`.

- ****Deduplication****: We use `ROW\_NUMBER() OVER(PARTITION BY ... ORDER BY ...)` to ensure the newest record is inserted, filtering out exact duplicates.

- ****Data Typing****: Fields like `heart\_rate` are ingested as `VARCHAR` to prevent `COPY INTO` from failing on bad data. During validation, `TRY\_CAST(heart\_rate AS NUMBER)` safely attempts conversion.

- **Quarantine**: If a conversion fails (returns NULL) or a business rule is broken (e.g., negative billing amount), the record is discarded from the main flow and inserted into ``governance.dq_exception_log``.

> [!TIP]

> This proves to stakeholders that the data pipeline is resilient. Bad data is actively logged rather than crashing the system!

4. Dimensional Modeling (CURATED)

The core of the data warehouse.

Surrogate Keys

We use ``UUID_STRING()`` to generate unique primary keys (e.g., ``patient_key``, ``appointment_key``) instead of relying on the source system's ``patient_id``. This protects the warehouse if the source system re-uses an ID.

Slowly Changing Dimensions (SCD Type 2)

Implemented on the ``dim_patient`` table.

- **The Problem**: If a patient moves from "New York" to "Seattle", simply updating their record destroys history. Previous hospital bills in New York would suddenly look like they happened in Seattle.

- **The Solution (SCD-2)**: The stored procedure ``curated.populate_curated_models()`` "expires" the old New York record by setting ``end_date = CURRENT_DATE()`` and ``is_current = FALSE``. It then inserts a brand new record for Seattle with ``is_current = TRUE``.

5. The Anomaly Engine (GOVERNANCE)

Data Quality checks (in the Validated layer) catch bad data structures. The **Anomaly Engine** catches bad **business processes**.

After curation, ``governance.run_anomaly_engine()`` scans the clean data for illogical operations:

1. **Double Booking**: Patient booked for two different appointments on the exact same day.
2. **Billing Outliers**: Uses statistical functions (``AVG`` and ``STDDEV``) to flag bills that are greater than 3 standard deviations from the department average.

6. Business Value Delivery (DATAMART & ANALYTICAL)

Instead of forcing BI analysts to join complex SCD-2 and Fact tables, we provide specialized views.

- * **PATIENT_360_MART**: Joins Facts and Dimensions to give a complete view of a patient's clinical and financial lifetime history.
- * **RISK_PREDICTION_MART**: Evaluates age, blood pressure, and cholesterol to classify patients into HIGH, MEDIUM, and LOW risk cohorts.
- * **REVENUE_TRENDS_MART**: Financial aggregates for the CFO.
- * **NO_SHOW_MART**: Operational efficiency metrics.

KPIs

The ``ANALYTICAL`` schema sits on top of the Datamarts to provide instant, single-row metrics for executive dashboards, such as the ``kpi_health_risk`` and ``kpi_data_quality`` views.

7. Orchestration (Tasks)

The entire system is glued together using a Snowflake Task DAG (Directed Acyclic Graph):

1. `task_process_dq`: Runs every 1 minute, but *only* if `SYSTEM$STREAM_HAS_DATA` is true.
2. `task_populate_curated`: Triggered automatically (`AFTER task_process_dq`).
3. `task_anomaly_engine`: Triggered automatically (`AFTER task_populate_curated`).

This ensures compute is only used exactly when needed, in the exact correct order.